

COPY Verb

The COPY Verb

The COPY verb is very different from other COBOL verbs.

While other COBOL statements are executed at run time the COPY is executed at compile time.

The COPY statement allows programs to include frequently used source code text from a copy file or copy library.

The COPY can include source code text without change or it can change the text as it is copied into the client program.



The COPY verb2

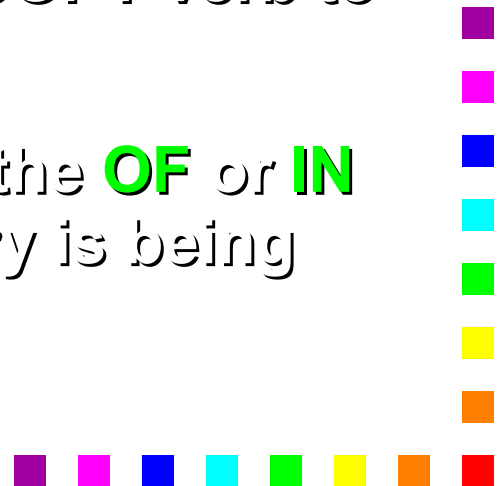
The COPY verb is generally used when creating large software systems.

It allows item descriptions to be kept and updated centrally in a copy Library, often under the control of a copy librarian.

A copy library contains COBOL elements that can be reference using a *textname*.

Each client program which wants to use items described in the copy library uses the COPY verb to include the descriptions it requires.

When more than one copy library is used the **OF** or **IN** qualifier is used to indicate which library is being referenced.



Why use the COPY verb?

Using the COPY verb makes implementation simpler by reducing the amount of coding required and eliminating transcription errors.

Using the COPY verb makes some maintenance tasks easier and safer.

The text in the copy library is updated and the affected programs are recompiled.

Using copy libraries makes it more difficult for programmers to make ad hoc changes to file and record formats.

Changes generally have to be approved by the COPY librarian.



COPY format

$$\underline{\text{COPY}} \left\{ \begin{array}{l} \text{TextName} \\ \text{ExternalFileNameLiteral} \end{array} \right\} \left[\left\{ \underline{\text{OF}} \right\} \left\{ \begin{array}{l} \text{LibraryName} \\ \text{LibraryNameLiteral} \end{array} \right\} \right]$$

$$\left[\left\{ \underline{\text{REPLACING}} \right\} \left\{ \begin{array}{l} \text{== PseudoText1 ==} \\ \text{Identifier1} \\ \text{Literal1} \\ \text{Word1} \end{array} \right\} \underline{\text{BY}} \left\{ \begin{array}{l} \text{== PseudoText2 ==} \\ \text{Identifier2} \\ \text{Literal2} \\ \text{Word2} \end{array} \right\} \right] \dots$$

Example COPY statements.

COPY "CopyFile2.CBL" **REPLACING** XYZ **BY** 120.

ExternalFileNameLiteral

COPY Copyfile3 **IN** "d:\visocd\examples\EGLIB".

TextName

LibraryNameLiteral

COPY "CopyFile3.CBL" **IN** "d:\visocd\examples\EGLIB".

ExternalFileNameLiteral

LibraryNameLiteral



How the COPY works

If the REPLACING phrase is not used then the compiler simply copies the text into the client program without change.

If the COPY does use the REPLACING phrase then the text is copied and each properly matched occurrence of Pseudo-Text-1, Identifier-1, Literal-1 and Word-1 in the library text is replaced by the corresponding Pseudo-Text-2, Identifier-2, Literal-2 or Word-2 in the REPLACING phrase.

Pseudo-Text is any COBOL text enclosed in double equal signs (e.g. ==ADD 1==).

It allows us to replace a *series* of words or characters as opposed to an individual identifier, literal or word.



How the REPLACING works

The REPLACING phrase tries to match text-words in the library text with text-words before the BY in the REPLACING phase.

If a match is achieved then as text is copied from the library it is replaced by the matching REPLACING phrase text.

For purposes of matching, each occurrence of a

separator comma

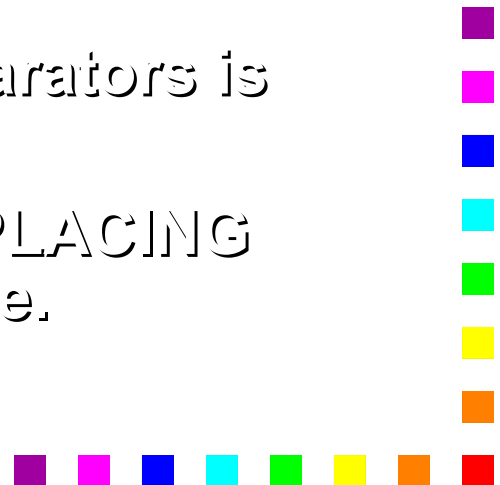
semicolon

space

in pseudo-text-1 or in the library text is considered to be a single space.

Each sequence of one or more space separators is considered to be a single space.

Comment lines in either the library or REPLACING phrase text are treated as a single space.



Text-Words

The matching procedure in the REPLACING phrase operates on text-words.

A text word is ;

A literal including opening and closing quotes

A separator other than;

A Space

A Pseudo-Text delimiter

A Comma

Opening and closing parentheses

Any other sequence of contiguous characters, bounded by separators.



Text-Word Examples

MOVE

1 Text Word

MOVE Total TO Print-Total

4 Text Words - MOVE Total TO Print-Total

MOVE Total TO Print-Total.

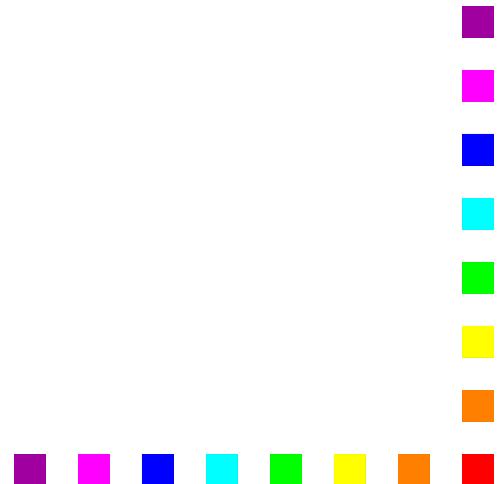
5 Text Words - MOVE Total TO Print-Total .

PIC S9(4)V9(6)

9 Text words - PIC S9 (4) V9 (6)

“PIC S9(4)V9(6)”

1 Text word - “PIC S9(4)V9(6)”



COPY Example 1

```
IDENTIFICATION DIVISION.  
PROGRAM-ID. COPYEG1.  
AUTHOR. Michael Coughlan.
```

```
ENVIRONMENT DIVISION.  
FILE-CONTROL.
```

```
    SELECT StudentFile ASSIGN TO "STUDENTS.DAT"  
    ORGANIZATION IS LINE SEQUENTIAL.
```

```
DATA DIVISION.  
FILE SECTION.  
FD StudentFile.  
COPY COPYFILE1.
```

```
PROCEDURE DIVISION.  
BeginProg.
```

```
    OPEN INPUT StudentFile  
    READ StudentFile
```

```
        AT END SET EndOfSF TO TRUE  
    END-READ
```

```
    PERFORM UNTIL EndOfSF
```

```
        DISPLAY StudentNumber SPACE StudentName SPACE  
            CourseCode SPACE FeesOwed SPACE AmountPaid
```

```
        READ StudentFile
```

```
            AT END SET EndOfSF TO TRUE
```

```
        END-READ
```

```
    END-PERFORM
```

```
    STOP RUN.
```

01 StudentRec.

88 EndOfSF VALUE HIGH-VALUES.

02 StudentNumber PIC 9(7).

02 StudentName PIC X(60).

02 CourseCode PIC X(4).

02 FeesOwed PIC 9(4).

02 AmountPaid PIC 9(4)V99.



COPY Example 2

Copyfile2.cbl

```
02 StudName PIC X(20) OCCURS XYZ TIMES.
```

```
01 NameTable2.  
COPY "CopyFile2.CBL" REPLACING XYZ BY 120.
```

```
01 NameTable2.  
    02 StudName PIC X(20) OCCURS 120 TIMES.
```

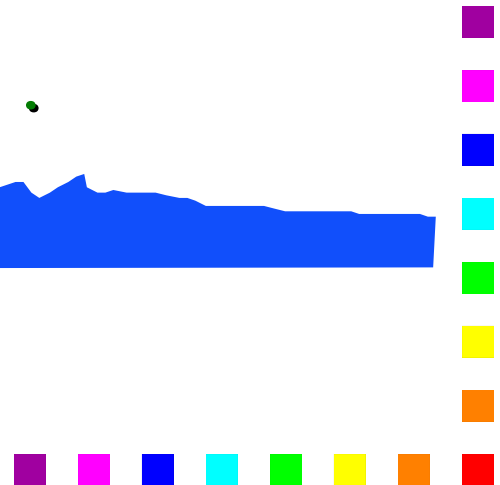
COPY Example 3

Copyfile3.cbl

```
02  CustOrder          PIC 9(R) .
```

```
01 CopyData.  
COPY Copyfile3 REPLACING ==R== BY ==4==.
```

```
01 CopyData.  
    02  CustOrder          PIC 9(4) .
```



COPY Example 4

Copyfile4.cbl

02 CustOrder2

PIC 9(6)V99.

01 CopyData.

COPY CopyFile4 REPLACING ==V99== BY ==.

01 CopyData.

02 CustOrder2

PIC 9(6).

COPY Example 5

Copyfile5.cbl

```
02 CustKey          PIC X(3) VALUE "KEY".
```

```
01 CopyData.  
COPY CopyFile5 REPLACING "KEY" BY "ABC".
```

```
01 CopyData.  
    02 CustKey          PIC X(3) VALUE "ABC".
```

COPY Example 6

Copyfile5.cbl

```
02 CustKey          PIC X(3) VALUE "KEY".
```

```
01 CopyData.  
COPY CopyFile5 REPLACING "CustKey" BY "Cust".
```

```
01 CopyData.  
    02 CustKey          PIC X(3) VALUE "KEY".  
                                No Replacement
```



COPY Example 7

Copyfile5.cbl

```
02 CustKey          PIC X(3) VALUE "KEY".
```

```
01 CopyData.  
COPY CopyFile5 REPLACING "KEY" BY == "ABC".  
    02 CustNum          PIC 9(8) ==.
```

```
01 CopyData.  
    02 CustKey          PIC X(3) VALUE "ABC".  
    02 CustNum          PIC 9(8)
```



COPY Example 8

Copyfile5.cbl

02 CustKey

PIC X(3) VALUE "KEY".

* the (is a textword replaced by @

COPY copyfile5 REPLACING (BY @.

* the 3 between (and) is a textword replaced by the textword "three"

COPY copyfile5 REPLACING 3 BY three.

* the) is a textword replaced by &

COPY copyfile5 REPLACING) BY &.

02 CustKey

PIC X@3) VALUE "KEY".

02 CustKey

PIC X(three) VALUE "KEY".

02 CustKey

PIC X(3& VALUE "KEY".

COPY Example 9

Copyfile5.cbl

```
02 CustKey PIC X(3) VALUE "KEY".
```

* the X before (3) is a textword replaced by pseudotext "Replace the X"

```
COPY copyfile5 REPLACING X  
BY ==Replace the X==.
```

* the pseudotext X(3) is replaced by X(19)

```
COPY copyfile5 REPLACING ==X(3)==  
BY ==X(19)==.
```

* the series of textwords X (3) is replaced by the series X (19)

```
COPY copyfile5 REPLACING X(3) BY X(19).
```

```
02 CustKey PIC Replace the X(3) VALUE KEY".
```

```
02 CustKey PIC X(19) VALUE "KEY".
```

```
02 CustKey PIC X(19) VALUE "KEY".
```

COPY Example 10

Copyfile5.cbl

```
02 CustKey PIC X(3) VALUE "KEY".
```

* the textword PIC is replaced by the pseudotext Pic is Replaced

```
COPY copyfile5 REPLACING PIC  
BY ==Pic is Replaced==.
```

* The P in PIC is not a textword by itself and so is not replaced

```
COPY copyfile5 REPLACING P BY  
==But P in PIC not replaced==.
```

```
02 CustKey Pic is Replaced X(3) VALUE "KEY".
```

```
02 CustKey PIC X(3) VALUE "KEY".
```

Not Replaced